# Student Record Management System

Data Structures & Algorithms Project • Language: C++17

*Data Structure: std::vector*


**Submitted By**

Muhammad Yousaf

**Course**

Analysis of Algorithm

**Instructor**

Maueed Ahmed Mirza

# Introduction

What is this system?

The Student Record Management System is a console-based application developed in C++17, designed to manage student records efficiently using dynamic arrays (std::vector).

## Core Operations

- **Add Student**
- **Display Records**
- **Search Student**
- **Delete Student**
- **Sorting**
- **GPA Statistics**

# Project Objectives

Goals of the System

| | |
|---|---|
| 01 | Implement a functional student management system |
| 02 | Apply Data Structures concepts in C++ |
| 03 | Analyze algorithms using RAM Model |
| 04 | Study Time Complexity and Space Complexity |
| 05 | Understand searching and sorting algorithms |

# Technologies Used

Tools and frameworks powering this project

| Component | Technology |
|---|---|
| Programming Language | C++17 |
| IDE | VS Code / Dev C++ |
| Data Structure | std::vector |
| Algorithms | Linear Search, Introsort |
| Concepts | RAM Model, Big O Notation |

# System Features

Main capabilities of the application

- Add Student Record
- Display All Students
- Search by ID
- Search by Name
- Delete Student
- Sort by GPA
- Sort by Name
- Sort by ID
- View GPA Statistics

# Data Structure: std::vector

Core data model of the system

```
struct Student {
    int id;
    string name;
    double gpa;
};
```

## Why std::vector?

- Dynamic Size
- Fast Random Access
- O(1) Amortized Insertion
- Better Memory Locality
- Easy Implementation

# System Architecture

Flow and module breakdown

User Input → Menu Driven System → Operations on Vector → Output Display

## Modules

- Student Data Model
- CRUD Operations
- Searching Module
- Sorting Module
- Statistics Module

# Searching Operations

Search by ID and Name

## Search by ID

- Uses Linear Search
- Best Case = O(1)
- Worst Case = O(n)

## Search by Name

- Case insensitive search
- Searches complete vector
- Worst Case = O(n)

# Sorting Operations

std::sort() — Introsort under the hood

## Introsort Combines:

**Quicksort**

For large partitions

**Heapsort**

When depth exceeds 2·log n

**Insertion Sort**

For small partitions

## Complexity

| Case | Complexity |
|---|---|
| Best Case | O(n log n) |
| Average Case | O(n log n) |
| Worst Case | O(n log n) |

# RAM Model Analysis

Counting primitive operations

**RAM Model counts primitive operations:**

| Read | Write | Compare | Add | Divide |
|---|---|---|---|---|

**Example: displayStats()**

$$T(n) = 3n + 1$$

- 3 operations per student (read id, read name, read gpa)
- 1 final division (compute average GPA)

# Complexity Analysis

Time complexity of each operation

| Operation | Complexity |
|---|---|
| Add Student | O(n) |
| Display All | O(n) |
| Search by ID | O(n) |
| Search by Name | O(n) |
| Delete Student | O(n) |
| Sort Operations | O(n log n) |
| Display Stats | O(n) |

# Space Complexity

Memory usage analysis

## Total Space Complexity = O(n)

$$O(n) + O(\log n) + O(1) = O(n)$$

Vector stores all student records, contributing O(n) space.

### Auxiliary Space Breakdown

| | |
|---|---|
| Vector (student records) | O(n) |
| Sorting stack depth (Introsort) | O(log n) |
| Most operations (CRUD) | O(1) |

# Sample Operations

Example data and outputs

**1. Add New Student**

**2. Search Student by ID**

**3. Sort Students by GPA**

**4. Display GPA Statistics**

**Example Record:**

```
ID: 1001    Name: Yousaf    GPA: 3.85
```

# Advantages of the System

What makes this system effective

- ✓ Simple and User Friendly
- ✓ Efficient Data Handling
- ✓ Easy Searching and Sorting
- ✓ Dynamic Memory Usage
- ✓ Good Performance for Small Datasets

# Limitations

Current constraints of the system

- ✗ Uses Linear Search (O(n) — inefficient for large data)
- ✗ No Database Integration
- ✗ Console Based Interface only
- ✗ Performance decreases for very large datasets

# Future Enhancements

Planned improvements and extensions

- Add Database Support (SQL)
- GUI Based Interface
- Login Authentication
- File Handling
- Binary Search Implementation
- Hash Table Integration

# Conclusion

Key takeaways from this project

**The project successfully demonstrates:**

- Practical use of Data Structures
- Searching and Sorting Algorithms
- Time and Space Complexity Analysis
- RAM Model Application

*The system is efficient, simple, and suitable for learning algorithm analysis concepts.*

# Thank You!

Questions?